```
package handlers

import (
 "io"
 "net/http"
 "strconv"
 "strings"

 "music-library-api/internal/dto"
 "music-library-api/internal/mappers"
 "music-library-api/internal/models"
 "music-library-api/internal/services"

 "github.com/gin-gonic/gin"
 "github.com/hajimehoshi/go-mp3"
 "go.mongodb.org/mongo-driver/mongo"
)

type TrackHandler struct {
 service services.ITrackService
 mongodb *mongo.Database
}

func NewTrackHandler(service services.ITrackService, mongodb *mongo.Database) *TrackHandler {
 return &TrackHandler{
  service: service,
  mongodb: mongodb,
 }
}

// GetTracks godoc
// @Summary      Get list of tracks
// @Description  Retrieve a paginated list of tracks
// @Tags         tracks
// @Accept       json
// @Produce      json
// @Param        page   query     int     false  "Page number"
// @Param        limit  query     int     false  "Page size"
// @Success      200    {object}  dto.TrackListResponse
// @Failure      500    {object}  map[string]string
// @Router       /tracks [get]
func (h *TrackHandler) GetTracks(c *gin.Context) {
 page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
 limit, _ := strconv.Atoi(c.DefaultQuery("limit", "10"))

 list, err := h.service.GetTracks(page, limit)
 if err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
  return
 }

 resp := make([]dto.TrackResponse, 0)
 for _, t := range list {
  resp = append(resp, mappers.ToTrackResponse(t))
 }
```

```
 c.JSON(http.StatusOK, dto.TrackListResponse{
  Page:  page,
  Limit: limit,
  Data:  resp,
 })
}

//
// ----------------------------
// GET /tracks/:id
// ----------------------------
//

// GetTrackByID godoc
// @Summary      Get track by ID
// @Description  Retrieve a single track by its ID
// @Tags         tracks
// @Produce      json
// @Param        id     path      string  true  "Track ID"
// @Success      200    {object}  dto.TrackResponse
// @Failure      404    {object}  map[string]string
// @Router       /tracks/{id} [get]
func (h *TrackHandler) GetTrackByID(c *gin.Context) {
 id := c.Param("id")

 track, err := h.service.GetTrackByID(id)
 if err != nil {
  c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
  return
 }

 c.JSON(http.StatusOK, mappers.ToTrackResponse(track))
}

// CreateTrack godoc
// @Summary      Upload a new track
// @Description  Upload a new MP3 track and save metadata
// @Tags         tracks
// @Accept       multipart/form-data
// @Produce      json
// @Param        artist       formData  string  true  "Artist"
// @Param        album        formData  string  false "Album"
// @Param        genre        formData  string  false "Genre"
// @Param        release_year formData  int     false "Release Year"
// @Param        file         formData  file    true  "MP3 File"
// @Success      201    {object} dto.TrackResponse
// @Failure      400    {object} map[string]string
// @Failure      500    {object} map[string]string
// @Router       /tracks [post]
func (h *TrackHandler) CreateTrack(c *gin.Context) {
 var req dto.TrackCreateRequest

 if err := c.ShouldBind(&req); err != nil {
  c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
```

```
 return
 }

 if !strings.HasSuffix(strings.ToLower(req.File.Filename), ".mp3") {
  c.JSON(http.StatusBadRequest, gin.H{"error": "only mp3 files allowed"})
  return
 }

 file, err := req.File.Open()
 if err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to open file"})
  return
 }
 defer file.Close()

 gridFSID, err := h.service.UploadMP3ToGridFS(req.File.Filename, file)
 if err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": "upload to GridFS failed"})
  return
 }

 file.Seek(0, 0)
 decoder, err := mp3.NewDecoder(file)
 if err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to decode mp3"})
  return
 }

 duration := int(float64(decoder.Length()/4) / float64(decoder.SampleRate()))

 track := &models.Track{
  Title:       req.File.Filename,
  Artist:      req.Artist,
  Album:       req.Album,
  Genre:       req.Genre,
  ReleaseYear: req.ReleaseYear,
  Duration:    duration,
  FileID:      gridFSID,
 }

 if err := h.service.CreateTrack(track); err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to save track"})
  return
 }

 c.JSON(http.StatusCreated, mappers.ToTrackResponse(track))
}

// UpdateTrack godoc
// @Summary      Update a track
// @Description  Update track metadata by ID
// @Tags         tracks
// @Accept       json
// @Produce      json
// @Param        id     path      string              true  "Track ID"
```

```
// @Param        track  body      dto.UpdateTrackRequest   true  "Track update info"
// @Success      200    {object}  dto.TrackResponse
// @Failure      400    {object}  map[string]string
// @Failure      404    {object}  map[string]string
// @Failure      500    {object}  map[string]string
// @Router       /tracks/{id} [patch]
func (h *TrackHandler) UpdateTrack(c *gin.Context) {
 id := c.Param("id")

 track, err := h.service.GetTrackByID(id)
 if err != nil {
  c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
  return
 }

 var req dto.UpdateTrackRequest
 if err := c.ShouldBindJSON(&req); err != nil {
  c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
  return
 }

 track.Title = req.Title
 track.Artist = req.Artist
 track.Album = req.Album
 track.Genre = req.Genre
 if req.ReleaseYear != 0 {
  track.ReleaseYear = req.ReleaseYear
 }

 if err := h.service.UpdateTrack(track); err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
  return
 }

 c.JSON(http.StatusOK, mappers.ToTrackResponse(track))
}

// DeleteTrack godoc
// @Summary      Delete a track
// @Description  Delete a track by ID
// @Tags         tracks
// @Param        id     path      string  true  "Track ID"
// @Success      204    "No Content"
// @Failure      404    {object}  map[string]string
// @Failure      500    {object}  map[string]string
// @Router       /tracks/{id} [delete]
func (h *TrackHandler) DeleteTrack(c *gin.Context) {
 id := c.Param("id")

 track, err := h.service.GetTrackByID(id)
 if err != nil {
  c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
  return
 }
```

```
 if err := h.service.DeleteTrack(track.ID.Hex()); err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
  return
 }

 c.Status(http.StatusNoContent)
}

// SearchTracks godoc
// @Summary      Search tracks
// @Description  Search tracks by query string
// @Tags         tracks
// @Produce      json
// @Param        q      query     string true  "Search query"
// @Param        page   query     int    false "Page number"
// @Param        limit  query     int    false "Page size"
// @Success      200    {object}  dto.TrackListResponse
// @Failure      500    {object} map[string]string
// @Router       /tracks/search [get]
func (h *TrackHandler) SearchTracks(c *gin.Context) {
 query := c.Query("q")
 page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
 limit, _ := strconv.Atoi(c.DefaultQuery("limit", "10"))

 list, err := h.service.SearchTracks(query, page, limit)
 if err != nil {
  c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
  return
 }

 resp := make([]dto.TrackResponse, 0)
 for _, t := range list {
  resp = append(resp, mappers.ToTrackResponse(t))
 }

 c.JSON(http.StatusOK, dto.TrackListResponse{
  Page:  page,
  Limit: limit,
  Data:  resp,
 })
}

// StreamTrack godoc
// @Summary Stream an audio track
// @Description Stream MP3 file, support Range header
// @Tags tracks
// @Produce audio/mpeg
// @Param id path string true "Track ID"
// @Header 206 {string} Content-Range "Bytes range of the content"
// @Header 206 {string} Content-Type "audio/mpeg"
// @Failure 404 {object} map[string]string
// @Router /tracks/{id}/stream [get]
func (h *TrackHandler) StreamTrack(c *gin.Context) {
 id := c.Param("id")
```

```
 track, err := h.service.GetTrackByID(id)
 if err != nil {
  c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
  return
 }

 rangeHeader := c.GetHeader("Range")

 stream, err := h.service.OpenTrackStream(track.FileID, rangeHeader)
 if err != nil {
  c.JSON(http.StatusRequestedRangeNotSatisfiable, gin.H{"error": err.Error()})
  return
 }
 defer stream.Reader.Close()

 c.Header("Content-Type", "audio/mpeg")
 c.Header("Accept-Ranges", "bytes")
 c.Header("Content-Disposition", "inline; filename=\""+track.Title+"\"")
 c.Header("Content-Range", stream.ContentRange)
 c.Status(http.StatusPartialContent)

 toCopy := (stream.End - stream.Start) + 1
 buf := make([]byte, 32*1024)
 var copied int64

 for copied < toCopy {
  n, err := stream.Reader.Read(buf)
  if n > 0 {
   remain := toCopy - copied
   if int64(n) > remain {
    n = int(remain)
   }
   c.Writer.Write(buf[:n])
   c.Writer.Flush()
   copied += int64(n)
  }

  if err != nil {
   if err == io.EOF {
    break
   }
   break
  }
 }
}
```